Girmiti Software — DevOps Engineer (AWS)

100 Interview Questions & Production-Focused Answers

Interview focus: AWS • Terraform • CloudFormation • CI/CD • Python/Shell • Networking • TLS/mTLS • Observability • PCI DSS • SOC 2 • Security • DR • Kubernetes • Production Troubleshooting

How to use this guide: Memorize the first 2–4 sentences of each answer as the spoken response. Expand only when the interviewer asks for implementation details. Keep answers grounded in your actual experience.

SECTION 1 — INTRODUCTION / PROFILE

Tell me about yourself.

Answer: I have 11+ years of experience across DevOps, SRE and cloud infrastructure, working primarily with AWS and Azure. My core experience is around production operations, Kubernetes, Docker, Terraform, CI/CD, monitoring, incident management and automation.

In my recent role, I worked on the Azure Synapse platform supporting production workloads and customer issues, including P1/P2 incidents, observability, SLI/SLO monitoring, alert optimization and RCA. On AWS, I've worked with services such as EC2, EKS, Lambda, networking, IAM, load balancing and RDS-related environments. I've also worked extensively with Terraform, Jenkins, GitHub Actions, Python and Shell automation.

I particularly focus on production reliability, automation, reducing operational toil and making deployments safer and more repeatable.

Why are you interested in this DevOps Engineer role?

Answer: The role matches my experience very closely because it combines AWS infrastructure, Terraform, CI/CD, production support, security, monitoring and automation. I also have experience working in production environments where reliability, incident response, security remediation and controlled deployments are important. The payment-domain exposure is particularly interesting because it requires stronger discipline around security, availability, change management and compliance.

What are your strongest DevOps areas?

Answer: My strongest areas are production troubleshooting, Terraform, Kubernetes and Docker, CI/CD, cloud infrastructure, monitoring and incident management. I'm also comfortable with Python and Shell scripting for operational automation. From an SRE perspective, I focus heavily on observability, SLI/SLO, alert quality, RCA and reducing repetitive operational work.

SECTION 2 — AWS

Explain a typical AWS production architecture you have worked with.

Answer: A typical architecture would have Route 53 for DNS, an ALB for HTTP/HTTPS traffic, application workloads running on EC2, ECS or EKS, IAM for access control, VPC with public and private subnets, security groups and route tables for network isolation, and RDS for the database layer. Monitoring would be handled through CloudWatch and other observability tools. Infrastructure would be provisioned through Terraform and deployments through CI/CD. For production, I would also consider multi-AZ availability, backups, autoscaling, logging, alerting, security controls and disaster-recovery requirements.

Difference between EC2, ECS and EKS?

Answer: EC2 provides virtual machines where I manage the operating system and application runtime. ECS is AWS's managed container orchestration service and is simpler when I want to run Docker workloads without managing Kubernetes. EKS is managed Kubernetes, where AWS manages the Kubernetes control plane while I manage workloads, nodes or node groups and Kubernetes configuration. I've worked with both EKS and containerized workloads and use the orchestration platform based on application and operational requirements.

ECS vs EKS — when would you choose each?

Answer: I would choose ECS when the requirement is primarily running containers and we want simpler AWS-native orchestration with lower Kubernetes operational overhead. I would choose EKS when we need Kubernetes capabilities such as portability, advanced scheduling, Kubernetes-native tooling, Helm, operators or a broader Kubernetes ecosystem. The decision should consider operational complexity, team expertise and application requirements rather than choosing Kubernetes automatically.

What is an Auto Scaling Group?

Answer: An Auto Scaling Group maintains the desired number of EC2 instances and can automatically add or remove instances based on policies or health conditions. For example, I could configure minimum, desired and maximum capacity and scale based on CPU utilization, request count or custom CloudWatch metrics. For production, I would combine it with a load balancer and health checks so unhealthy instances are automatically replaced.

ALB vs NLB?

Answer: ALB operates at Layer 7 and is designed for HTTP and HTTPS traffic. It supports host-based and path-based routing. NLB operates at Layer 4 and is designed for high-performance TCP, UDP or TLS traffic and provides very high throughput and low latency. For a normal web application requiring URL-based routing, I would typically use ALB. For TCP-level workloads or specific performance requirements, NLB may be more appropriate.

What happens when a request reaches an AWS application?

Answer: DNS resolves the application through Route 53. The request reaches the load balancer, typically an ALB, which performs listener and routing-rule evaluation. It forwards the request to healthy targets such as EC2 instances or containers. The application then accesses downstream services such as RDS, Redis or other APIs through the VPC network. Throughout the path I would monitor latency, errors, target health, network connectivity and application logs.

SECTION 3 — AWS VPC / NETWORKING

Explain VPC architecture.

Answer: I normally design a VPC across multiple Availability Zones. Public subnets are generally used for internet-facing components such as load balancers, while application and database workloads are placed in private subnets. Internet Gateway provides internet connectivity for public resources. Private resources requiring outbound internet access can use a NAT Gateway. Route tables determine traffic paths, while security groups and network ACLs provide additional traffic controls.

Security Group vs NACL?

Answer: Security groups are stateful and attached to resources such as EC2 instances or ENIs. If inbound traffic is allowed, the response traffic is automatically allowed. Network ACLs operate at the subnet level and are stateless, so inbound and outbound rules need to be considered independently. I normally use security groups as the primary resource-level access control and NACLs for additional subnet-level restrictions where required.

An EC2 instance cannot connect to RDS. How do you troubleshoot?

Answer: First I determine the impact and confirm whether the issue affects one instance or multiple workloads. Then I check RDS availability and endpoint, security-group rules, EC2 security group, RDS subnet and route tables, network ACLs, DNS resolution, port connectivity, credentials/application configuration, and recent infrastructure or security changes. I would use nslookup, dig, nc -vz or application-specific database connectivity tests and correlate with CloudWatch and application logs before making changes.

Application cannot access the internet from a private subnet. What do you check?

Answer: I would check whether the private subnet has a route to a NAT Gateway, whether the NAT Gateway is healthy and whether the NAT Gateway is located in a subnet with a route to an Internet Gateway. Then I would verify route tables, security groups, NACLs and DNS resolution. I would also verify that the application isn't blocked by proxy, firewall or endpoint-specific restrictions.

What is a NAT Gateway?

Answer: NAT Gateway allows resources in private subnets to initiate outbound internet connections without exposing those resources directly to inbound internet traffic. Private EC2 instances may need to download packages or call external APIs and can route outbound traffic through NAT while remaining inaccessible directly from the internet.

How would you troubleshoot intermittent network connectivity?

Answer: I first establish whether it's application-specific, host-specific, AZ-specific or network-wide. Then I check latency, packet loss, DNS resolution, connection resets, load balancer metrics, security changes, route tables and network infrastructure metrics. I correlate timestamps across application logs, CloudWatch and infrastructure metrics. Once isolated, I mitigate customer impact first and then perform RCA.

SECTION 4 — IAM / SECURITY

How do you follow least privilege in AWS?

Answer: I grant only the permissions required for a specific workload or operation. I prefer IAM roles instead of long-lived access keys wherever possible. I separate human and workload access, use resource-level permissions when supported and periodically review unused permissions. For production, I also use logging and auditing to identify unexpected access.

IAM Role vs IAM User?

Answer: An IAM user represents a long-term identity and can have credentials. An IAM role provides temporary credentials and is generally preferred for workloads and cross-account access. For EC2, ECS or other AWS workloads, I prefer attaching an appropriate IAM role rather than storing static AWS access keys on the server.

How would you secure AWS production workloads?

Answer: I would use private subnets where appropriate, least-privilege IAM, security groups, encryption at rest and in transit, centralized logging, vulnerability scanning, patch management and controlled CI/CD deployments. I would also monitor CloudTrail and CloudWatch and ensure secrets aren't hardcoded in source code or pipeline configurations.

SECTION 5 — TERRAFORM

Explain your Terraform experience.

Answer: I've used Terraform for infrastructure provisioning across AWS and Azure. I have worked with reusable modules, variables, environment-specific configurations, Terraform state and CI/CD integration. My focus is making infrastructure repeatable, idempotent and reviewable through Git-based workflows. For production changes, I prefer a workflow where Terraform plan is reviewed before apply, with appropriate approvals and state protection.

What is Terraform state?

Answer: Terraform state maintains Terraform's representation of the infrastructure resources it manages. It allows Terraform to determine what currently exists and what changes are required. In production I prefer remote state with locking and controlled access rather than storing state locally.

What problems can occur with Terraform state?

Answer: State corruption, concurrent modifications, accidental deletion, state drift, incorrect resource imports and unauthorized access are common risks. I mitigate them through remote backend storage, locking, access controls, versioning, backup and controlled CI/CD execution.

What is Terraform drift?

Answer: Drift occurs when infrastructure is changed outside Terraform, so the actual infrastructure differs from Terraform's state or configuration. I identify it using terraform plan and then determine whether the external change should be imported into the desired configuration or reverted.

What are Terraform modules?

Answer: Modules are reusable Terraform components that encapsulate infrastructure patterns. For example, instead of repeatedly creating VPCs, security groups or EC2 configurations, I can create standardized modules and provide environment-specific inputs. This improves consistency and reduces duplication.

terraform plan vs terraform apply?

```
terraform plan calculates and displays the proposed infrastructure changes without applying them. terraform apply e
```

How do you prevent accidental production infrastructure changes?

Answer: I use Git-based change control, pull requests, Terraform plan review, approval gates, protected branches and restricted production credentials. Production state should also have controlled access and locking. I avoid allowing engineers to make undocumented manual changes unless it's an emergency, and emergency changes should subsequently be reconciled back into IaC.

SECTION 6 — CLOUDFORMATION

What is AWS CloudFormation?

Answer: CloudFormation is AWS's native Infrastructure-as-Code service. Infrastructure is defined using templates, traditionally YAML or JSON, and CloudFormation creates and manages the resources as a stack. The major benefit is that AWS resources and their dependencies can be managed consistently and reproducibly.

Terraform vs CloudFormation?

Answer: Terraform is cloud-agnostic and supports multiple providers, whereas CloudFormation is AWS-native. CloudFormation integrates deeply with AWS services, while Terraform provides a broader multi-cloud ecosystem and a declarative workflow that I'm very familiar with. Since my stronger hands-on experience is with Terraform, that's where I have spent most of my infrastructure automation work.

SECTION 7 — CI/CD

Explain your CI/CD pipeline.

Answer: A typical pipeline starts with Git commit and pull request validation. Then we build the application using tools such as Maven or Gradle, run unit tests and security or quality checks, create an artifact or container image and publish it to an artifact repository. The deployment stage promotes the artifact through environments using controlled approvals. For production, I include validation, health checks, monitoring and rollback mechanisms.

How would you design a production deployment pipeline?

Answer: I would separate build from deployment so the same immutable artifact is promoted across environments. The pipeline would include: Git → Build → Unit Test → Security Scan → Package → Artifact Repository → Deploy → Health Check → Monitoring → Approval → Production. For critical workloads I would use blue-green or canary deployment and automatic rollback based on health metrics.

What is artifact immutability?

Answer: Once an artifact is built and tested, I don't rebuild it separately for each environment. The same versioned artifact is promoted from development to staging and production. This ensures that what was tested is exactly what gets deployed.

What is the difference between Maven and Gradle?

Answer: Both are build automation tools commonly used for Java applications. Maven uses a convention-based XML configuration and has a very standardized lifecycle. Gradle uses a more flexible DSL and generally provides more customization and incremental build capabilities. Operationally, my focus is integrating either tool reliably into CI/CD with testing, artifact creation and repository publishing.

What is Nexus?

Answer: Nexus is an artifact repository manager. It can store Maven packages, Docker images and other build artifacts. In CI/CD, the pipeline can publish versioned artifacts to Nexus and later retrieve the same artifact during deployment.

Bamboo vs Jenkins?

Answer: Both provide CI/CD automation. Jenkins has a large plugin ecosystem and is highly extensible. Bamboo provides strong integration within the Atlassian ecosystem. My stronger hands-on experience is with Jenkins and GitHub Actions, but the core CI/CD concepts are transferable.

SECTION 8 — PYTHON / SHELL

How have you used Python in DevOps?

Answer: I use Python mainly for operational automation, API integration, data processing and troubleshooting utilities. For example, I can automate repetitive operational tasks, query APIs, process logs or build utilities that integrate with infrastructure workflows.

Write a Shell script to check disk usage.

Answer: Use a threshold and inspect filesystem usage:

```
THRESHOLD=80
```

Answer:

```
df -P | awk 'NR>1 {print $5, $6}' | while read usage mount
do
    usage=${usage%\%}
```

Answer: if ["$usage" -ge "$THRESHOLD"]; then

```
        echo "WARNING: $mount is ${usage}% full"
    fi
done
```

In production I would also add logging, alerting, exclusions for temporary mounts if required, and proper exit codes.

How would you find the largest files?

Answer: Use:

```
find /var -type f -printf '%s %p\n' 2>/dev/null | sort -nr | head -10
```

I would first identify the filesystem and then investigate whether the files are logs, application data or temporary files before deleting anything.

How would you restart a service if it is down?

Answer: Use:

```
if ! systemctl is-active --quiet nginx; then
    systemctl restart nginx
fi
```

In production I would not blindly restart it. I would first check service status, logs and recent changes and make sure restart is an appropriate mitigation.

SECTION 9 — TLS / CERTIFICATES

Explain TLS.

Answer: TLS provides encryption and authentication between communicating systems. During the TLS handshake, the client and server negotiate cryptographic parameters and the server presents a certificate. The client validates the certificate chain, hostname and validity before establishing the encrypted session.

What is mTLS?

Answer: In normal TLS, the server authenticates itself to the client. In mutual TLS, both sides authenticate each other using certificates. This is useful for service-to-service communication, partner APIs and other environments where both parties need strong identity verification.

What happens if a TLS certificate expires?

Answer: The client may reject the connection because the certificate is outside its validity period. This can cause API failures, service-to-service communication failures or customer-facing outages. I would verify certificate expiry, certificate chain, hostname, truststore and server configuration. For prevention, I would implement certificate-expiry monitoring and automated or controlled rotation.

What is a truststore?

Answer: A truststore contains certificates or certificate authorities that a client trusts when validating a remote server certificate.

What is a keystore?

Answer: A keystore typically contains a private key and corresponding certificate used by an application or server to identify itself during TLS communication.

How would you troubleshoot a TLS handshake failure?

Answer: I would first identify whether the failure is certificate validation, protocol mismatch, cipher incompatibility, hostname mismatch, expired certificate or missing CA chain. I would inspect application logs and use tools such as openssl s_client to inspect the certificate chain and handshake details. Then I would verify the truststore, certificate chain and server configuration.

Example:

```
openssl s_client -connect example.com:443 -servername example.com
```

SECTION 10 — MONITORING / OBSERVABILITY

What metrics would you monitor for an AWS application?

Answer: I would monitor the Golden Signals: latency, traffic, errors and saturation. At infrastructure level I would monitor CPU, memory, disk, network, instance health and load-balancer metrics. At application level I would monitor request rate, error rate, response time, dependency failures and business-critical metrics.

CloudWatch vs Prometheus?

Answer: CloudWatch is deeply integrated with AWS and provides infrastructure and AWS-service metrics, logs and alarms. Prometheus is particularly strong for application and Kubernetes metrics using a pull-based model and PromQL. In a mixed AWS/Kubernetes environment, I can use both: CloudWatch for AWS-native telemetry and Prometheus/Grafana for application and Kubernetes observability.

How do you avoid alert fatigue?

Answer: I design alerts around actionable conditions rather than every metric anomaly. I use thresholds based on symptoms and SLO impact where possible, tune evaluation windows, remove duplicate alerts and define severity levels. I also periodically review alert-to-incident correlation and identify noisy alerts that don't result in meaningful action.

What is an SLI?

Answer: SLI is a quantitative measurement of service behavior, such as availability, latency or error rate.

What is an SLO?

Answer: SLO is the target value for an SLI. For example, an availability SLO could be 99.9%.

What is an SLA?

Answer: SLA is a formal agreement with customers or stakeholders that can include availability and service commitments. SLO is an internal reliability target, while SLA usually has contractual or business implications.

Explain error budget.

Answer: Error budget is the amount of unreliability permitted by the SLO. For a 99.9% availability target, the remaining 0.1% represents the error budget. If we consume the budget rapidly, we should prioritize reliability work and be more cautious with risky releases.

SECTION 11 — PRODUCTION TROUBLESHOOTING

Production application is down. What do you do?

Answer: First I determine the customer impact and severity. Then I check monitoring dashboards, load-balancer health, application health, infrastructure metrics and recent deployments or changes. I correlate logs and metrics to isolate whether the problem is application, infrastructure, database, network or dependency related. My priority is mitigation and service restoration first. Once service is stable, I perform RCA, identify the contributing factors and implement preventive actions.

Mental framework: Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent.

CPU suddenly reaches 100%. What do you do?

Answer: I first identify whether it is one instance or a fleet-wide problem. Then I check CPU trends, processes, application logs, traffic levels, deployment history and autoscaling behavior. If a process is consuming CPU because of abnormal traffic or a runaway process, I mitigate based on the situation — scaling, traffic control or restarting the affected workload if appropriate. After recovery I perform RCA rather than treating the restart as the solution.

Memory is continuously increasing.

Answer: I would determine whether this is a memory leak, traffic increase, cache growth or expected workload behavior. I check memory metrics, process-level memory usage, container limits, application logs and recent deployments. If containers are being OOMKilled, I verify the Kubernetes or container memory limits and requests and investigate application behavior before simply increasing the limit.

RDS CPU is 95%. What do you investigate?

Answer: I would check active connections, query performance, slow queries, connection pool behavior, read/write workload, locks and recent application changes. I would use CloudWatch metrics and database-level performance information to determine whether the issue is query-related, capacity-related or traffic-related. Depending on the cause, mitigation could include query optimization, connection-pool correction, scaling or read-replica usage.

Application latency suddenly increases.

Answer: I first establish whether latency is at the application, load-balancer, database or downstream dependency layer. I compare latency with traffic, error rate and saturation. Then I inspect traces or logs where available and correlate the timestamp with deployments, infrastructure changes and dependency behavior. I mitigate the affected path first and then investigate the underlying cause.

SECTION 12 — SECURITY / VULNERABILITY MANAGEMENT

How do you handle a critical vulnerability in production?

Answer: First I validate the vulnerability and determine the affected assets and exposure. Then I assess exploitability and business impact and identify an appropriate remediation such as patching, upgrading a dependency, changing configuration or applying a compensating control. I test the remediation in a controlled environment, deploy through the approved change process and validate afterward. Finally, I document evidence and close the vulnerability with the appropriate security team.

How do you integrate security into CI/CD?

Answer: I would integrate multiple security gates into the pipeline: SAST → Dependency Scan → Container Scan → IaC Scan → Secret Detection → Build → Deploy. Critical findings can block production deployment depending on the organization's policy and risk classification.

What is the difference between SAST and DAST?

Answer: SAST analyzes source code or compiled code to identify vulnerabilities without executing the application. DAST tests the running application from an external perspective to identify runtime vulnerabilities.

How would you secure Docker images?

Answer: I use minimal trusted base images, pin versions, avoid running as root where possible, scan images for vulnerabilities, remove unnecessary packages and secrets, and rebuild regularly to pick up security patches. I also ensure only approved images are deployed into production.

SECTION 13 — PCI DSS

What is PCI DSS?

Answer: PCI DSS is a security standard for organizations that store, process or transmit payment card data. It focuses on areas such as access control, network security, vulnerability management, secure configuration, logging, monitoring and protection of cardholder data.

What DevOps responsibilities are relevant to PCI DSS?

Answer: From a DevOps perspective, I would focus on secure infrastructure configuration, least-privilege access, vulnerability remediation, patching, logging, monitoring, encryption, controlled deployments and maintaining evidence of changes. Infrastructure changes should be traceable through version control and approved change processes.

What kind of evidence could an auditor ask for?

Answer: Depending on the control, evidence could include access-control records, IAM reviews, vulnerability scan results, patch records, change approvals, deployment logs, security-group configuration, CloudTrail logs, monitoring evidence, incident records, and backup/restore evidence. The important part is that the evidence should demonstrate that the control exists and operates consistently.

What is SOC 2?

Answer: SOC 2 is an assurance framework focused on controls related to areas such as security, availability, processing integrity, confidentiality and privacy. From an engineering perspective, this can involve access control, change management, monitoring, incident management, risk management and evidence collection.

If asked about direct SOC 2 experience: My direct documented exposure has been stronger around production security, PCI-related remediation and operational controls. I understand the SOC 2 control concepts and evidence requirements, but I would not claim ownership of a complete SOC 2 audit unless I had directly led one.

SECTION 14 — BACKUP / DISASTER RECOVERY

What is RPO?

Answer: Recovery Point Objective defines how much data loss is acceptable, expressed as a time interval. For example, an RPO of 15 minutes means the organization aims to recover with no more than approximately 15 minutes of data loss.

What is RTO?

Answer: Recovery Time Objective defines how quickly a service needs to be restored after an outage.

How would you design DR for an AWS application?

Answer: I would start with business requirements such as RTO and RPO. Then I would determine the appropriate strategy: backup and restore, pilot light, warm standby or active-active. I would consider database replication, application deployment automation, DNS failover, infrastructure-as-code, data backups and monitoring. Most importantly, I would regularly test the recovery process rather than assuming that backups alone guarantee recoverability.

How do you validate backups?

Answer: I don't consider a backup successful just because the backup job reports success. I would periodically perform restoration tests, validate data integrity, measure recovery time and document the results against RTO/RPO requirements.

SECTION 15 — COST OPTIMIZATION

How would you reduce AWS cost?

Answer: First I identify the major cost drivers using AWS Cost Explorer and billing reports. Then I look at underutilized EC2 instances, excessive storage, unused resources, data-transfer costs, oversized RDS instances and non-production resources running unnecessarily. Depending on workload, optimization could include right-sizing, autoscaling, scheduling non-production resources, storage lifecycle policies and appropriate purchasing models.

How do you balance cost and reliability?

Answer: I don't optimize cost in isolation. The first question is the business reliability requirement. For example, reducing multi-AZ redundancy might save money but could violate availability requirements. I look for waste first — unused resources, overprovisioning, inefficient storage and unnecessary data transfer — before compromising reliability.

SECTION 16 — CHANGE MANAGEMENT

How do you handle a production change?

Answer: I start with a clearly defined change, risk assessment, implementation plan, validation plan and rollback plan. The change goes through the required review and approval process. During implementation I monitor the relevant health indicators and validate the application afterward. If unexpected impact occurs, I execute the rollback or mitigation plan and document the outcome.

A deployment caused an outage. What do you do?

Answer: First I stabilize the service. If the deployment is clearly correlated with the outage and rollback is safe, I would roll back to the last known-good version. Then I verify service health and customer impact. After recovery, I investigate the failure, identify why testing or controls didn't catch it and implement preventive measures such as stronger automated testing, canary deployment or additional health checks.

SECTION 17 — DOCKER / CONTAINERS

Docker container keeps restarting. How do you troubleshoot?

Answer: I check the container status and exit code first, then inspect logs and configuration. I check environment variables, secrets, mounted volumes, ports, dependencies and resource limits. If it's Kubernetes, I also check pod events, readiness/liveness probes, resource requests and limits and recent deployment changes.

Difference between CMD and ENTRYPOINT?

Answer: ENTRYPOINT defines the primary executable for the container, while CMD provides default arguments or a default command. They can also be combined, where ENTRYPOINT defines the executable and CMD supplies default arguments.

SECTION 18 — KUBERNETES

Pod is in CrashLoopBackOff. What do you do?

Answer: I start with:

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
```

Then I check events, configuration, secrets, probes, resource limits, dependencies and recent deployment changes. --previous is particularly useful because the current container may have already restarted.

Pod is Pending.

Answer: I check:

```
kubectl describe pod <pod>
```

Then I look at scheduler events for insufficient CPU/memory, node selectors, taints/tolerations, affinity rules, PVC issues or unavailable nodes. I then determine whether the problem is resource capacity, scheduling constraints or storage.

Pod is OOMKilled.

Answer: I check the container's memory usage and configured memory limit. Then I determine whether the application genuinely needs more memory or whether there's abnormal memory growth. I check recent

changes, traffic patterns and application behavior before increasing the limit.

SECTION 19 — AI-ASSISTED DEVOPS

How would you responsibly use AI in DevOps?

Answer: I would use approved AI tools for tasks such as generating initial scripts, analyzing logs, creating documentation, explaining Terraform or Kubernetes errors and accelerating troubleshooting. However, I would validate generated code before execution, avoid exposing secrets or sensitive production information and follow organizational security policies. AI should accelerate engineering work, not replace engineering review.

SECTION 20 — BEHAVIORAL / SENIOR ENGINEER

Tell me about a production incident you handled.

Answer: In my recent SRE role, I handled production P1/P2 issues affecting customers. My approach was to first establish the impact and scope, then use monitoring dashboards, logs and service metrics to isolate the affected component. I coordinated with the relevant engineering teams, focused first on mitigation and service restoration, and then performed RCA. I also looked for preventive actions such as alert tuning, better dashboards, runbooks or automation so the same issue would be less likely to recur.

Tell me about an automation you implemented.

Answer: One example was automating repetitive operational activities that previously required manual execution. I used automation through tools such as Rundeck/API-based workflows and scripting to standardize the process. The objective was to reduce manual effort and operational risk while making the execution repeatable and auditable.

How do you handle pressure during a P1?

Answer: I stay focused on impact and restoration rather than trying to immediately find the perfect root cause. I establish ownership, communicate clearly, divide investigation areas among engineers and continuously update stakeholders. Once service is restored, I move into RCA and preventive actions.

How do you communicate with non-technical stakeholders during an incident?

Answer: I avoid unnecessary technical detail. I communicate four things: current impact, what we're doing, expected next update and whether there are any customer or business risks. For example: "We have identified elevated API failures affecting a subset of customers. Engineering is working on mitigation. The next update will be in 15 minutes."

SECTION 21 — SCENARIO QUESTIONS

Customer API suddenly returns 401 errors.

Answer: I would first establish whether all customers or a subset are affected. Then I would check authentication configuration, IAM or application credentials, token expiry, recent deployments and configuration changes. If the API uses certificates or mTLS, I would also validate certificate expiry, trust chain and client authentication. I would correlate application and load-balancer logs and restore service through the safest mitigation.

API works internally but not from a partner network.

Answer: I would suspect connectivity, firewall, security-group, routing, DNS or TLS-related differences. I would compare the successful internal path with the failing partner path. Then I would validate DNS resolution, source IP restrictions, ports, routing, certificates and mTLS configuration. I would use connection and TLS diagnostics to isolate the layer causing the failure.

Deployment succeeds but application health check fails.

Answer: I would first verify whether the application process started successfully. Then I would inspect logs, health-check configuration, ports, environment variables, dependencies and readiness/liveness configuration. If the new version is unhealthy and rollback is safe, I would restore the previous version and investigate the new release separately.

Terraform pipeline wants to destroy a production resource.

Answer: I would stop the deployment and not blindly approve it. First I would inspect the Terraform plan and determine why Terraform believes the resource needs replacement or deletion. I would check recent code changes, state drift, provider changes and resource lifecycle configuration. Only after understanding the cause and confirming the intended outcome would I proceed through the appropriate production approval process.

SECTION 22 — RAPID-FIRE

What is idempotency?

Answer: Running the same automation multiple times produces the same desired final state without unintended side effects.

What is blue-green deployment?

Answer: Two production environments exist; traffic is switched from the old environment to the new environment after validation.

What is canary deployment?

Answer: A new version is initially exposed to a small percentage of traffic before gradually increasing exposure.

What is immutable infrastructure?

Answer: Infrastructure is replaced rather than manually modified in place, reducing configuration drift.

What is Infrastructure as Code?

Answer: Managing infrastructure through version-controlled declarative or programmatic definitions rather than manual configuration.

What is configuration drift?

Answer: The actual environment differs from the expected configuration defined by code or policy.

What is CloudTrail?

Answer: AWS service that records API activity and account actions for auditing and investigation.

What is CloudWatch?

Answer: AWS monitoring and observability service providing metrics, logs, alarms and dashboards.

What is Route 53?

Answer: AWS managed DNS and traffic-routing service.

What is an AMI?

Answer: An Amazon Machine Image is a template used to launch EC2 instances.

What is EBS?

Answer: Block storage attached to EC2 instances.

What is S3?

Answer: AWS object storage used for scalable and durable storage of objects such as files, backups and artifacts.

What is RDS?

Answer: Managed relational database service supporting engines such as PostgreSQL, MySQL and others.

What is a VPC endpoint?

Answer: It provides private connectivity from a VPC to supported AWS services without requiring traffic to traverse the public internet.

What is the difference between public and private subnet?

Answer: A public subnet has a route to an Internet Gateway, while a private subnet does not provide direct inbound internet connectivity.

Final 15 Questions to Prioritize

1. Tell me about yourself
2. Explain AWS production architecture
3. Explain VPC
4. EC2/ECS/EKS differences
5. Troubleshoot EC2 → RDS connectivity
6. Terraform state/drift/modules
7. Design CI/CD pipeline
8. Production incident troubleshooting
9. TLS/mTLS troubleshooting
10. Certificate expiry/rotation
11. PCI DSS from DevOps perspective
12. Vulnerability remediation
13. CloudWatch/observability
14. AWS cost optimization
15. Backup/DR + RPO/RTO

Interview positioning: Senior production-focused DevOps/SRE engineer with 11+ years of experience across AWS and Azure, strong Terraform/CI-CD/Kubernetes/container experience, strong incident and observability background, and practical security/PCI remediation exposure.

Important: For CloudFormation, SOC 2, or specific certificate/keystore tooling, distinguish your direct hands-on experience from transferable knowledge. Do not claim ownership you did not have.